

IMPACT ANALYSIS

CR-001: Forgot Password and Password Reset Feature

1. Change Summary

CR-001 introduces a secure Forgot Password and Password Reset feature to the Student Management System.

Users who forget their password will be able to initiate a password-reset request using their registered email address. The system will provide a secure password-reset mechanism that allows the user to create a new password.

The change is primarily related to authentication and account recovery. Existing login, authorization, student management, and administrator functionality must remain operational.

2. Impact Assessment

Area	Impact Assessment
Frontend	Moderate - New Forgot Password and Reset Password interfaces and validation logic will be added.
Backend	Moderate - Password-reset request and password-reset processing may require backend logic.
API	Moderate - New password-reset endpoints may be required if equivalent endpoints do not already exist.
Database	Low - May require temporary reset-token storage or additional password-reset fields.
Authentication	Moderate - Existing authentication workflow will be extended with password recovery.
Authorization	Low - Existing authorization must remain unchanged.

Security	High - Secure token generation, expiration, password hashing, and token invalidation are required.
Performance	Low - Password-reset operations are expected to occur only when users request them.
CRUD	None - Existing student CRUD functionality should remain unchanged.
Deployment	Moderate - Configuration for password-reset functionality and email service may be required.

3. Frontend Impact

The main affected area is the existing authentication interface.

Potential changes include:

- Forgot Password link on the Login page.
- Forgot Password request page.
- Email address input.
- Email validation.
- Password Reset page.
- New password input.
- Confirm password input.
- Password validation messages.
- Reset success message.
- Invalid/expired token message.
- Back to Login functionality.
- Responsive styling.

The existing Login interface and authentication workflow must continue to work correctly.

The password-reset interface must not expose sensitive information such as passwords or reset tokens unnecessarily.

4. Backend, API and Database Impact

Unlike CR-001, this change may require backend modifications because password resetting is a security-sensitive operation and should not be trusted to client-side logic alone.

Potential backend functionality includes:

- Password-reset request processing.
- User/email lookup.
- Secure reset-token generation.
- Reset-token expiration.
- Reset-token validation.
- Password update.
- Password hashing.
- Reset-token invalidation.

Potential API endpoints may include functionality equivalent to:

POST /api/auth/forgot-password

POST /api/auth/reset-password

These endpoints should only be introduced if equivalent functionality does not already exist.

The existing login endpoint should remain unchanged unless modification is technically required.

Database Impact

Depending on the existing database design, password-reset functionality may require temporary reset-token information.

Potential information may include:

- User ID.
- Reset-token hash.
- Token expiration time.
- Token creation time.
- Token-used status/time.

No changes should be made to unrelated student data or student-management tables.

Any database modification must be documented in the final implementation report.

5. Authentication and Security Impact

This change has a **moderate to high security impact** because it introduces an additional method for obtaining access to an existing account.

Existing authentication and authorization mechanisms must remain protected.

The password-reset feature must:

- Not bypass normal authentication security.
- Generate secure reset tokens.
- Use an appropriate token expiration period.
- Reject invalid tokens.
- Reject expired tokens.
- Prevent reset-token reuse.
- Securely hash new passwords.
- Never store passwords in plain text.

- Avoid exposing passwords in logs or error messages.
- Avoid unnecessarily revealing whether an email address is registered.
- Prevent unauthorized users from resetting another user's password.
- Use secure transport such as HTTPS in deployment.
- Use normal safe React rendering.
- Preserve existing administrator authorization.
- Preserve existing backend security controls.

The password-reset feature must not provide access to an account merely because a user knows another user's email address.

6. CRUD and Functional Impact

The Forgot Password feature should have **no direct impact on Student CRUD functionality**.

Existing administrator functionality must continue to operate correctly, including:

- Create student.
- View student.
- Edit student.
- Delete student.
- Search/filter student list where applicable.

The password-reset change must not modify student records or student CRUD logic.

Regression testing should confirm that implementing the authentication change does not break existing administrator functionality.

7. Performance Impact

The expected performance impact is **low**.

Password-reset operations occur only when a user requests account recovery and therefore should not significantly affect normal system operation.

The implementation should avoid:

- Unnecessary database queries.
- Repeated reset-token generation.
- Unnecessary API requests.
- Expensive processing during normal login.

The email/reset mechanism should be implemented so that password-reset processing does not negatively affect normal application performance.

If the system experiences a large number of password-reset requests in the future, rate limiting or additional background processing may be considered as a separate enhancement.

8. Testing Impact

Testing should cover functional, security, validation, and regression scenarios.

Manual testing should cover:

- Forgot Password link.
- Forgot Password page.
- Valid email submission.
- Invalid email format.
- Unknown email address.
- Password-reset mechanism.
- Valid reset token.

- Invalid reset token.
- Expired reset token.
- Reuse of reset token.
- New password entry.
- Confirm password validation.
- Weak password validation.
- Valid password reset.
- Login using the new password.
- Rejection of the old password.
- Password hashing.
- Account enumeration protection.
- Existing administrator login.
- Existing administrator authorization.
- Existing student CRUD functionality.
- Existing Student Management System functionality.
- Responsive Forgot Password interface.

9. Risk Analysis

Risk	Impact	Mitigation
Unauthorized user resets another user's password	High	Use secure, unpredictable, time-limited reset tokens.
Reset token is reused	High	Invalidate token immediately after successful password reset.
Reset token remains valid too long	High	Apply a defined token expiration period.
Password stored in plain text	Critical	Securely hash passwords before database storage.

Account enumeration through Forgot Password	Medium	Return a generic response that does not unnecessarily reveal account existence.
Weak password accepted	Medium	Enforce existing password policy.
Existing login functionality breaks	High	Perform authentication regression testing.
Existing administrator authorization is affected	High	Preserve existing authorization middleware/security controls.
Sensitive information exposed in logs	High	Do not log passwords or unnecessary reset-token information.
Invalid/expired token accepted	High	Perform server-side token validation.
Email/reset service unavailable	Medium	Display an appropriate user-facing response and document configuration requirements.
Forgot Password UI is difficult to use	Low	Perform usability and responsive-interface testing.
Password reset affects unrelated student data	Medium	Limit database updates to the intended user account.

10. Overall Impact and Recommendation

Overall Impact: Moderate

CR-001 is primarily an authentication and security enhancement. Unlike a purely frontend change, the Forgot Password feature may require coordinated frontend, backend, and potentially database changes.

The change is technically feasible and recommended for implementation, subject to formal approval and security testing.

The key implementation considerations are:

- Secure password-reset token generation.

- Appropriate token expiration.
- One-time token usage.
- Secure password hashing.
- Protection against unauthorized password changes.
- Protection against unnecessary account enumeration.
- Preservation of existing authentication and authorization.
- Regression testing of existing Student Management System functionality.

The implementation should be reviewed and tested carefully because password-reset functionality directly affects account security.

Recommendation: Proceed with implementation after formal approval, with particular emphasis on security testing and regression testing.